

Лабораторна робота № 9

СТРІЧКИ.

Мета роботи:

Вивчити поняття і застосування стрічок.

Обладнання та програмне забезпечення:

- IBM сумісна персональна обчислювальна машина;
- онлайн компілятор мови програмування Cі, доступний за посиланням <https://repl.it/languages/c>

Завдання до роботи:

Написати програму на мові програмування Cі, котра використовує наступні дії над стрічками:

- пошук заданої послідовності символів;
- заміну і видалення символів і груп символів;
- порівняння і сортування стрічок;
- знаходження кількості символів у стрічках.

Теоретичні відомості

Стрічки

В Cі **стрічка** визначається як символьний масив довільної довжини, що закінчується нульовим символом. Нульовий символ визначається як '\0'.

Тому необхідно оголошувати символьні масиви на один символ більші.

Наприклад, якщо необхідно оголосити масив s, що містить десятисимвольну стрічку, слід написати:

```
char s[11];
```

Стрічкові константи це список символів, взятих в подвійні лапки. Нижче показані дві стрічкові константи:

```
"Hello world"  
"This is a string constant"
```

Для роботи зі стрічками використовується модуль **string.h** стандартної бібліотеки мови Cі [1, 2]

Константи і типи модуля **string.h**:

NULL - нульовий (порожній) вказівник, який нікуди не вказує. Використовується для того, щоб показати, що дана змінна-вказівник ні на що не посилається.

size_t - беззнакове ціле, що має той же тип, що і результат оператора sizeof

Функції модуля **string.h**:

Прототип	Пояснення
<code>void *memcpy(void *dest, const void *src, size_t n);</code>	копіює <i>n</i> байтів з області пам'яті <i>src</i> в <i>dest</i> , які не повинні перетинатися, у іншому випадку результат невизначений (можливо як правильне копіювання, так і ні)
<code>void *memmove(void *dest, const void *src, size_t n);</code>	копіює <i>n</i> байтів з області пам'яті <i>src</i> в <i>dest</i> , які на відміну від <i>memcpy</i> можуть перекриватися
<code>void *memchr(const void *s, char c, size_t n);</code>	повертає вказівник на перше входження символу <i>c</i> в перших <i>n</i> байтах <i>s</i> , або NULL, якщо не знайдено
<code>int memcmp(const void *s1, const void *s2, size_t n);</code>	порівнює перші <i>n</i> символів в областях пам'яті <i>s1</i> та <i>s2</i>
<code>void *memset(void *, int z, size_t n);</code>	заповнює область пам'яті одним байтом <i>z</i>
<code>char *strcat(char *dest, const char *src);</code>	дописує стрічку <i>src</i> в кінець <i>dest</i>
<code>char *strncat(char *dest, const char *, size_t n);</code>	дописує не більше <i>n</i> початкових символів рядка <i>src</i> (або всю <i>src</i> , якщо її довжина менше) у кінець <i>dest</i>
<code>char *strchr(const char *s, int c);</code>	шукає символ <i>c</i> у рядку <i>s</i> , починаючи з голови і повертає його адресу, або NULL якщо не знайдений
<code>char *strrchr(const char *s, int c);</code>	шукає символ <i>c</i> у рядку <i>s</i> , починаючи з хвоста і повертає його адресу, або NULL якщо не знайдений
<code>int strcmp(const char *s1, const char *s2);</code>	лексикографічне порівняння стрічок (повертає "0", якщо стрічки однакові, додатне, якщо перша стрічка більша, і від'ємне, якщо менша)
<code>int strncmp(const char *s1, const char *s2, size_t n);</code>	лексикографічне порівняння перших <i>n</i> байтів стрічок
<code>int strcoll(const char *s1, const char *s2);</code>	лексикографічне порівняння двох стрічок відповідно до поточної локалі, визначеної категорією LC_COLLATE.
<code>char *strcpy(char *to, const char *from);</code>	копіює стрічку з одного місця в інше
<code>char *strncpy(char *to, const char *from, size_t n);</code>	копіює до <i>n</i> байт стрічки з одного місця в інше
<code>char *strerror(int);</code>	повертає в стрічковому вигляді повідомлення про помилку errno
<code>size_t strlen(const char *);</code>	повертає довжину стрічки
<code>size_t strspn(const char *s, const char *accept);</code>	повертає к-сть початкових байтів стрічки <i>s</i> , які містяться в стрічці <i>accept</i>
<code>size_t strcspn(const char *s, const char *reject);</code>	повертає к-сть початкових байтів стрічки <i>s</i> , які НЕ містяться в стрічці <i>reject</i>
<code>char *strpbrk(const char *s, const char *accept);</code>	знаходить перше входження у стрічку <i>s</i> будь-якого символу, перерахованого в <i>accept</i>
<code>char *strstr(const char *haystack, const char *needle);</code>	знаходить перше входження рядка <i>needle</i> в <i>haystack</i>
<code>char *strtok(char *, const char *);</code>	перетворює стрічку у послідовність токенів.
<code>size_t strxfrm(char *dest, const char *src, size_t n);</code>	створює відтрансльовану копію рядка, таку, що дослівне порівняння її (strcmp) буде еквівалентно порівнянню з коллатором

Приклад №1

```
#include <string.h>
#include <stdio.h>
int main(void)
{
    char s1[80], s2[80];
    scanf("%s", s1); scanf("%s", s2);
    // визначення довжин 2 стрічок
    printf("lengths: %lu %lu\n", strlen(s1), strlen(s2));
    if(!strcmp(s1, s2)) printf("The strings are equal\n");
    // об'єднання 2 стрічок
    strcat(s1, s2);
    printf("%s\n", s1);
    return 0;
}
```

Приклад №2

```
#include <string.h>
#include <stdio.h>
int main(void)
{
    char s1[80], s2[80];
    scanf("%s", s1); // ввід слова
    for(int i=0; i<strlen(s1);++i)
        printf("%c ", s1[i]); // посимвольний вивід слова
    printf("\n");
    for(int i=strlen(s1); i>0;--i)
        printf("%c ", s1[i]); // посимвольний вивід слова в зворотному порядку
    return 0;
}
```

Приклад №3

```
#include <stdio.h>
int main(void)
{
    int *p, mas[10];
    p = mas;
    mas[5] = 5;
    p[5] = 100; // Присвоєння за допомогою індексу
    printf("%d\n", mas[5]);
    *(p + 5) = 20; // Присвоєння за вказівником
    printf("%d\n", mas[5]);
    return 0;
}
```

ПОРЯДОК ВИКОНАННЯ РОБОТИ:

1. Опрацювати і засвоїти матеріал наведений в теоретичних відомостях.
2. Написати програму, яка реалізує введення із консолі прислів'я або афоризму із 7+ слів однією стрічкою, розбиває її на масив окремих слів із зворотнім порядком символів в словах і виводить їх в окремих рядках.
3. Написати програму, яка реалізує пострічкове введення із консолі списку Вашої групи (улюбленої футбольної команди, будь-якого колективу з 10+ учасників) в форматі Номер-[пробіл]-Прізвище-[пробіл]-Ім'я, а потім 1) підраховує і виводить кількість відмінних (різних) імен та середню довжину прізвищ; 2) формує і виводить в алфавітному порядку список виду Номер-[пробіл]-Ім'я-[пробіл]-Прізвище.
4. Написати програму, яка перевіряє на правильність введену із консолі адресу електронної пошти. Правильний формат: префікс@домен1.домен2[.домен3].
Приклади: 1) правильні – johndeer@gmail.com, ivan.franko@lnu.edu.ua;
2) неправильні – jackdoe@gmail, Іван2020@укр.нет.
5. В звіті навести копії екранів та написаний код.
6. Зробити висновки.